Instituto Tecnológico de Aeronáutica - ITA
CM-204 - Aprendizado Profundo
Aluno: Ulisses Lopes da Silva

Relatório do Laboratório 02 - Aprendizado Produndo com PyTorch

---

1 Caso teste e discussão dos resultados

O caso teste consiste em uma rede neural *feedforward* com arquitetura $2 \to 50 \to 1$, funções de ativação sigmóide nas camadas oculta e de saída, otimizador Adam ($lr = 10^{-3}$), *loss-function* `BCELoss()`, 5000 épocas e *batch size* de 50 ($n/4$). A rede é treinada para classificar pontos (x_1, x_2) gerados aleatoriamente numa região quadrada do plano, segundo duas funções alvo: `sum_gt_zero` ($x_1 + x_2 \geq 0$) e `xor`. As quatro combinações analisadas resultam do cruzamento das duas funções alvo com dois valores do hiperparâmetro de regularização L_2: $\lambda_{L_2} = 0{,}000$ e $\lambda_{L_2} = 0{,}002$.

1.1 Figuras comprovando o funcionamento do código

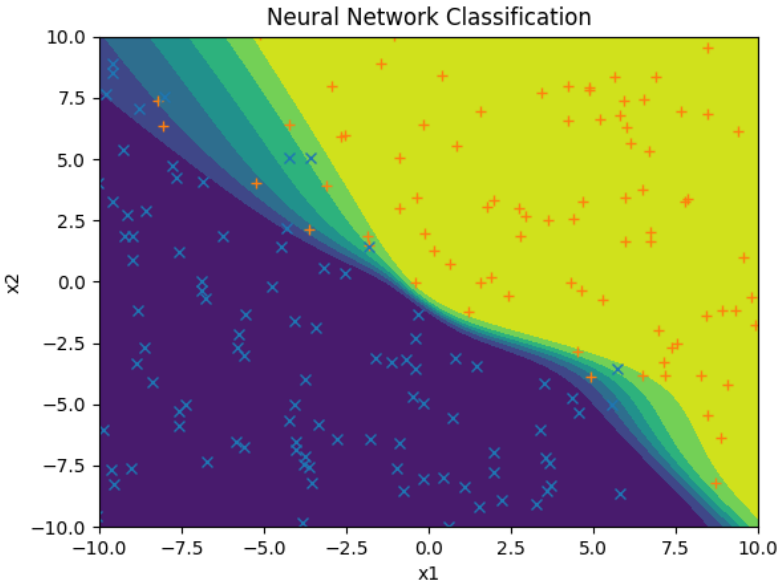

Fig. 1: Limite de decisão para $\lambda_{L_2} = 0{,}000$ e função `sum_gt_zero`.

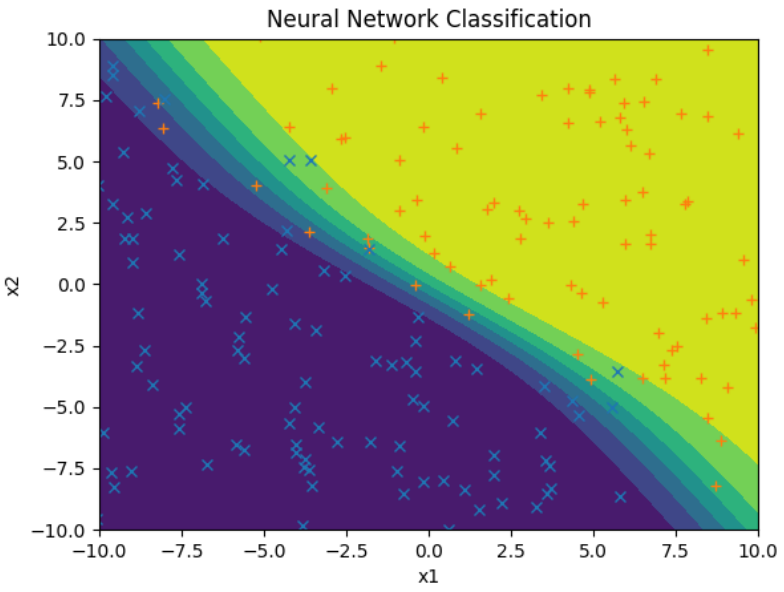

Fig. 2: Limite de decisão para $\lambda_{L_2} = 0{,}002$ e função `sum_gt_zero`.

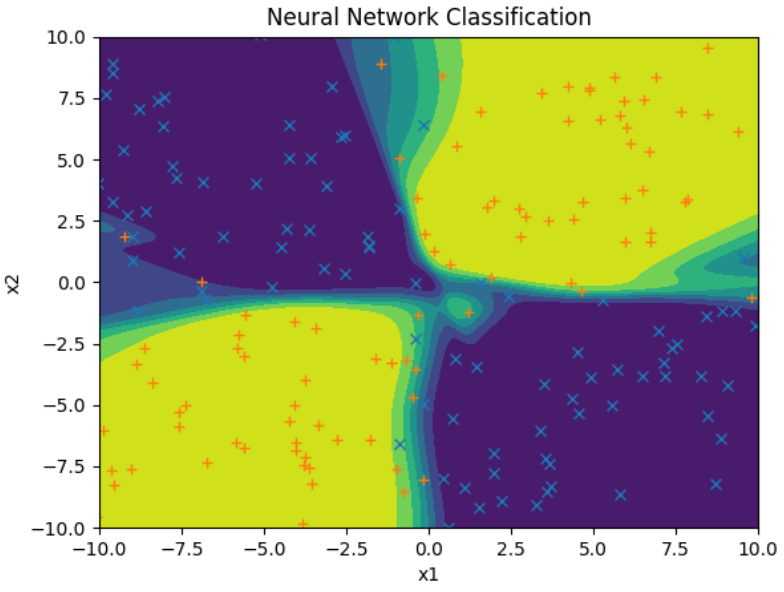

Fig. 3: Limite de decisão para $\lambda_{L_2} = 0{,}000$ e função `xor`.

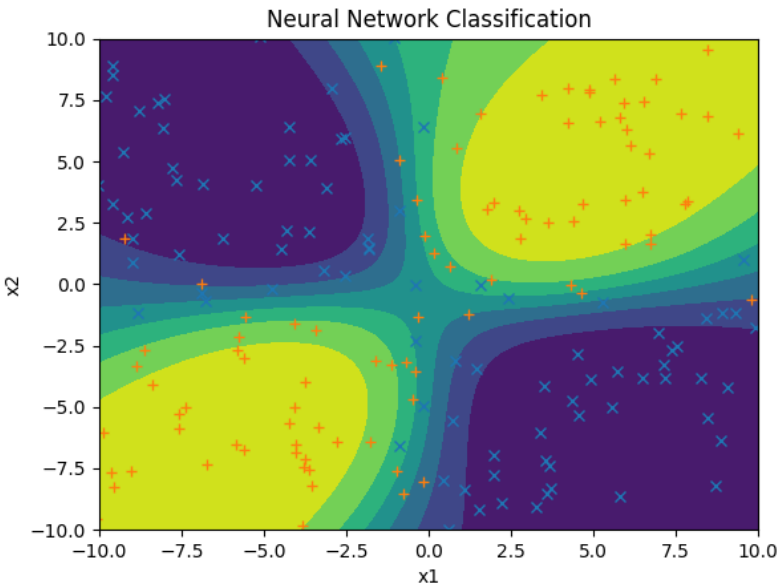

Fig. 4: Limite de decisão para $\lambda_{L_2} = 0{,}002$ e função `xor`.

1.2 Discussão dos Resultados

Para a função `sum_gt_zero`, as duas configurações produzem fronteiras de decisão consistentes com o problema, que é *linearmente separável* em torno da reta $x_1 + x_2 = 0$. Sem regularização (Fig. 1), a fronteira aprendida é levemente sinuosa, sugerindo que a rede se ajusta parcialmente ao ruído do conjunto de treinamento. Com $\lambda_{L_2} = 0{,}002$ (Fig. 2), os pesos são penalizados e a fronteira torna-se aproximadamente uma reta, mais próxima da solução que melhor aproxima o problema. Isto é coerente com o efeito esperado da regularização L_2, que reduz a variância do modelo ao desencorajar pesos de magnitude elevada.

Para a função `xor`, o problema é *não linearmente separável* e exige fronteiras de decisão com curvatura para isolar os quatro quadrantes alternados. Sem regularização (Fig. 3), a rede consegue separar as quatro regiões, embora as fronteiras apresentem irregularidades locais (como as próximas à origem) decorrentes do ajuste a pontos ruidosos. Com $\lambda_{L_2} = 0{,}002$ (Fig. 4), as fronteiras tornam-se sensivelmente mais suaves e simétricas, evidenciando o trade-off clássico viés-variância, em que a penalização nos pesos reduz a flexibilidade do modelo (aumenta o viés), mas elimina oscilações e melhora a generalização.

Em síntese, a regularização L_2 atua como um mecanismo de controle de complexidade efetivo nas duas funções testadas, sendo seu benefício mais perceptível visualmente no caso `xor`, em que a margem para sobreajuste é maior devido à não linearidade da fronteira ótima do problema.

2 Breve Explicação em Alto Nível da Implementação

A tarefa foi implementada como um problema de regressão supervisionada, em que, dada a entrada escalar t (instante do ciclo de caminhada, em segundos), a rede deve produzir um vetor

com as posições angulares das 20 juntas do robô humanoide naquele instante. O conjunto de treinamento corresponde ao primeiro ciclo da caminhada (40 amostras espaçadas de 8 ms).

A arquitetura da rede implementada segue exatamente a Tabela 1 abaixo, conforme o enunciado do problema que constava no arquivo do código. A camada de saída é *linear* (sem ativação), apropriada ao caráter contínuo e potencialmente negativo das posições angulares (em radianos):

Camada	**Dim. entrada → saída**	**Ativação**
`Linear` 1	$1 \to 75$	*Leaky* ReLU ($\alpha = 0{,}01$)
`Linear` 2	$75 \to 50$	*Leaky* ReLU ($\alpha = 0{,}01$)
`Linear` 3	$50 \to 20$	Linear

Tabela 1: Arquitetura da rede neural utilizada no *imitation learning*.

O treinamento utiliza o otimizador `Adam` com taxa de aprendizado 10^{-3}, sem regularização (`weight_decay` = 0, conforme exigido pelo enunciado), e a função de perda é o erro quadrático médio (`nn.MSELoss`). Conforme orientação, todo o *dataset* é processado em cada iteração, o que é obtido configurando `batch_size = len(dataset)` no `DataLoader`. A Tabela 2 consolida os hiperparâmetros.

Hiperparâmetro	**Valor**
Otimizador	Adam
Taxa de aprendizado	10^{-3}
Função de perda	MSE
Batch size	`len(dataset)`
Épocas	30000
Regularização L_2	não há

Tabela 2: Hiperparâmetros do treinamento.

A função `train()`, já fornecida no código base, foi reutilizada para o treinamento da rede pelo número de épocas acima definido, bem como para o *forward pass*, o cálculo da perda, o *backpropagation* e a atualização dos parâmetros. Após o treinamento, a rede é avaliada para gerar as trajetórias preditas, que são então comparadas com as trajetórias originais nas cinco juntas da perna direita.

3 Figuras Comprovando Funcionamento do Código

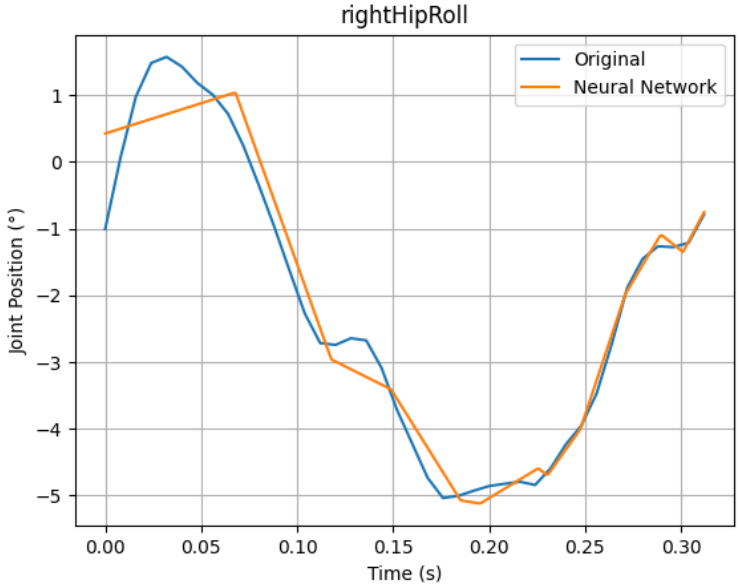

Fig. 5: Comparação entre trajetória original e predita pela rede neural para a junta `rightHipRoll`.

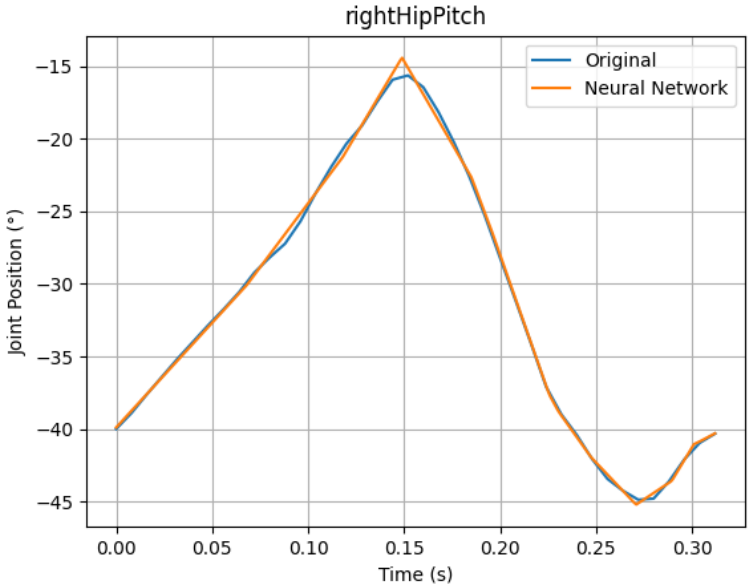

Fig. 6: Comparação entre trajetória original e predita pela rede neural para a junta `rightHipPitch`.

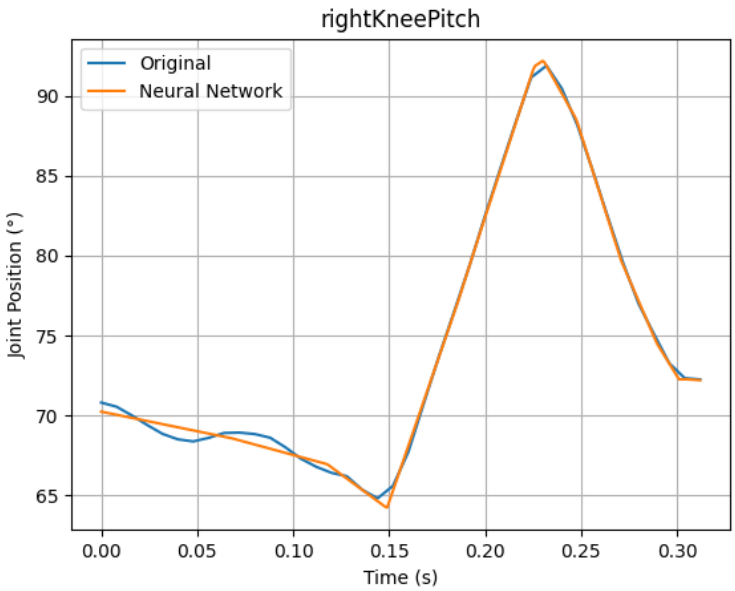

Fig. 7: Comparação entre trajetória original e predita pela rede neural para a junta `rightKneePitch`.

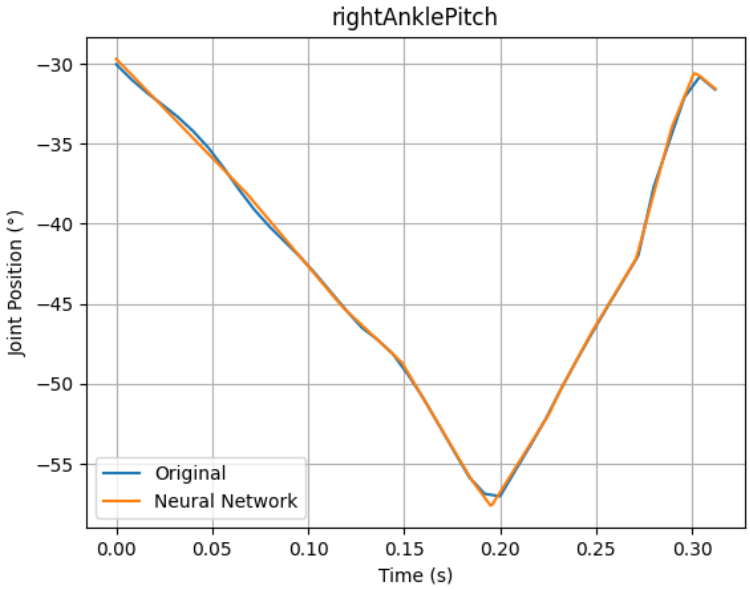

Fig. 8: Comparação entre trajetória original e predita pela rede neural para a junta `rightAnklePitch`.

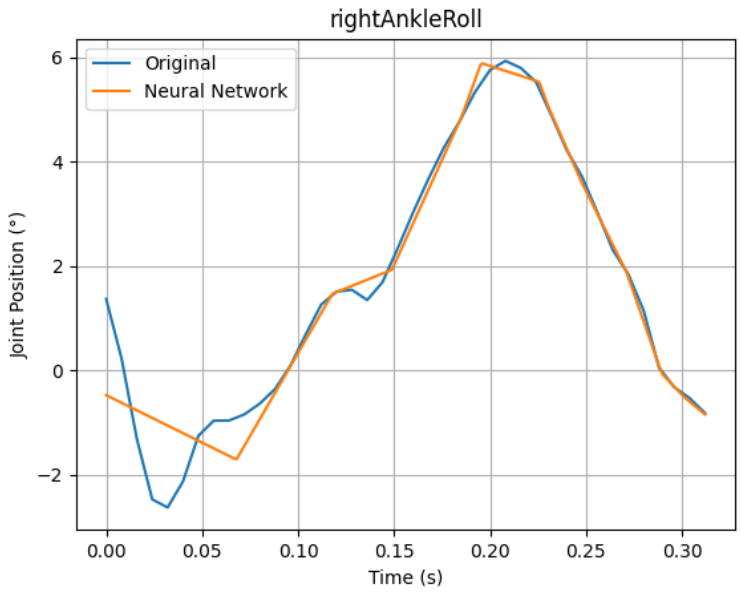

Fig. 9: Comparação entre trajetória original e predita pela rede neural para a junta `rightAnkleRoll`.

4 Discussão dos Resultados

A análise das figuras mostra que a rede neural é capaz de reproduzir, com boa fidelidade, as trajetórias originais das cinco juntas da perna direita ao longo do ciclo de caminhada. O ajuste é particularmente preciso nas juntas de maior amplitude angular — `rightKneePitch`, `rightHipPitch` e `rightAnklePitch` —, nas quais a curva predita praticamente sobrepõe a original em todo o período de captura do movimento.

Nas juntas de menor amplitude — `rightAnkleRoll` e `rightHipRoll` — observa-se uma leve suavização das oscilações presentes no início do ciclo. Esse comportamento é consistente com o viés indutivo da arquitetura: redes *feedforward* com ativações *Leaky* ReLU tendem a aprender preferencialmente componentes de baixa frequência da função alvo. Como o sinal absoluto dessas oscilações é pequeno, seu peso relativo na perda MSE total é reduzido, e o otimizador prioriza ajustar as juntas dominantes em magnitude.

Apesar dessa pequena imprecisão pontual, o resultado global é satisfatório para a tarefa de *imitation learning* proposta. A rede generaliza relativamente bem do ciclo de caminhada e reproduz o padrão coordenado das juntas da perna, comprovando que a abordagem de aprendizado supervisionado é viável para copiar uma política de controle previamente projetada por técnicas analíticas.